

Access Revocation Flow — System Design

This design describes the flow from a reviewer requesting revocation of an account's access right to the platform confirming removal from the target system and recording the outcome.

1. Scope and Assumptions

Access reviews support two decisions: **approve**, which retains existing access, and **revoke**, which initiates removal. This design focuses on revocation.

The reviewer's decision and the execution outcome are separate. Recording REVOKED acknowledges the request; it does not mean access has already been removed.

Assumptions:

- Each review item identifies an account, access right, and target system.
- Reviewers are authenticated and authorized for the review item.
- Targets may remove access immediately or process requests asynchronously.
- Target adapters define how removal is confirmed and whether repeated requests are safe.

2. End-to-End Revocation Flow

The platform uses a **database-backed job queue**: each revocation creates a persistent job that a background worker executes independently of the reviewer's request.

Background execution lets the platform acknowledge the reviewer's request without waiting for the target system. Because the job is stored in the database, it survives application restarts and can be scheduled for later execution if the target is unavailable.

The worker communicates with each target through an adapter that translates the platform's removal request into the target-specific API call and interprets its response. The worker then records confirmed completion, schedules a retry or confirmation check, or flags the job for intervention, as illustrated in Figure 1. Details of the steps follow below.

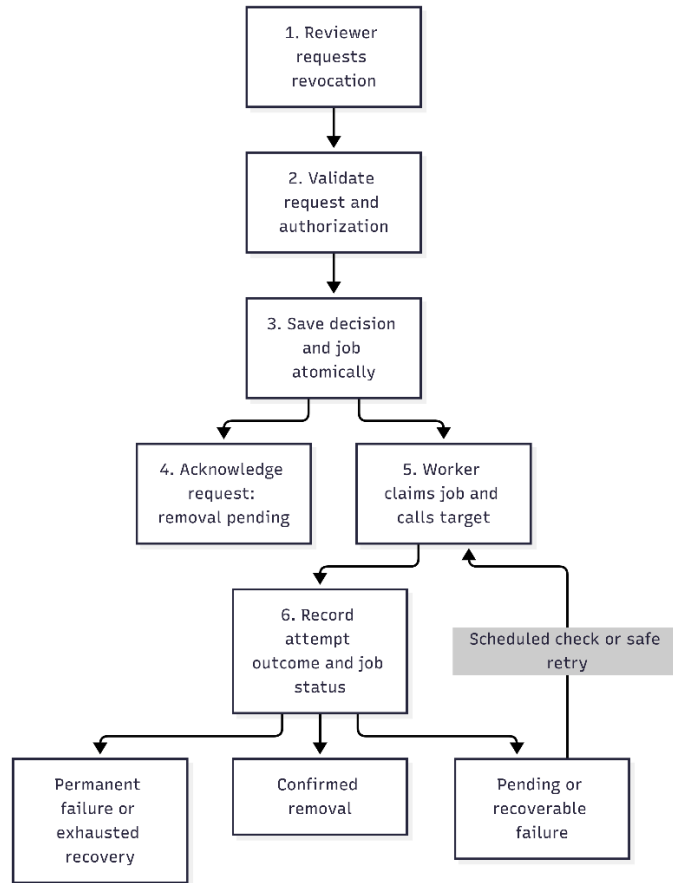


Figure 1. High-level diagram flow of the proposed solution

1. **A reviewer requests a revocation.** The flow starts.
2. **Validate the request.** Resolve the reviewed access assignment and verify the reviewer's authorization. A unique request ID prevents duplicate submissions from creating duplicate work. Conflicting decisions for the same review item are rejected.
3. **Persist the decision and job.** In one database transaction, record the decision and create a QUEUED revocation job. This prevents a saved revocation decision from being left without work scheduled to execute it. Approval records the decision without creating a job.
4. **Acknowledge request.** After the transaction commits, return the decision ID, job ID, and pending status. The reviewer can retrieve execution status using the job ID.
5. **Execute in the background.** A worker atomically claims a due job, records the attempt, and calls the target adapter. The network call occurs outside the database transaction.
6. **Persist the outcome.** Save the attempt result and update the job together. Confirmed removal completes the job; asynchronous acceptance schedules verification; failures trigger retry or intervention, according to section 4.

3. States and Persistence

Three records separate intent, current progress, and execution history:

Record	Information stored
Review decision	Review item, reviewer, decision, request ID, and timestamp
Revocation job	Decision reference, account/right/target identifiers, status, next action, next execution time, retry counters, processing deadline, claim token/expiration, and target operation ID
Revocation attempt	Job reference, action, timestamps, outcome, sanitized error details, target correlation ID, and confirmation evidence

Table 1. Records or entities used in the database for supporting the job queue

Each revoked decision has one job, enforced by a unique constraint on the decision reference. Each job can have multiple attempts. Status checks and removal attempts are counted separately.

Job state	Meaning
QUEUED	Waiting for initial execution
IN_PROGRESS	Claimed by a worker
PENDING_CONFIRMATION	Target accepted the request; completion is unconfirmed
RETRY_SCHEDULED	Another attempt or verification is scheduled
SUCCEEDED	Removal or absence of the intended access is confirmed
FAILED	Automatic processing stopped; intervention is required

Table 2. Different Job states across the workflow, and what they mean

A job starts as QUEUED and becomes IN_PROGRESS when claimed. After execution, it transitions to SUCCEEDED, PENDING_CONFIRMATION, RETRY_SCHEDULED, or FAILED, depending on the outcome. Pending and retry jobs become eligible for another claim when their next execution time arrives.

The job's status describes its current progress, while its next action tells the worker what to do: request removal, check a target operation, or verify whether access remains. This distinction prevents a pending confirmation from being treated as a new removal request.

Workers query due jobs using an index on status and next execution time. Claiming is atomic and assigns a time-limited lease with a new claim token. Result updates must match that token, preventing stale workers from overwriting newer progress.

FAILED describes the workflow outcome. After an unresolved timeout, the actual access state may still be unknown.

4. Failure Handling and Recovery

The target adapter classifies responses so the worker can choose a logical next action.

Scenario	Handling
Removal confirmed	Mark SUCCEEDED and retain evidence. An unambiguous confirmation that the intended access is already absent also counts as success.
Accepted asynchronously	Store the operation ID and schedule status checks. If no operation ID exists, use an authoritative access-state check when available.
Temporary failure	Schedule a safe retry with bounded exponential backoff and jitter. Persist the next execution time rather than sleeping in a worker.
Rate limited	Honor the target's retry delay. Coordinate request-rate and concurrency limits across workers sharing the same target quota.
Timeout or uncertain outcome	Check the operation or access state, or retry using a stable idempotency key when supported. Do not assume the request failed.
Permanent failure	Record the reason, mark FAILED, and surface for intervention.
Worker crash	Recover the job after its lease expires. Treat interrupted removal as uncertain and verify or retry safely.

Table 3. Different scenarios and consistent actions to handle them.

Failure classification depends on the target's documented behavior, not only its response code. A server error may occur after removal was applied, so the adapter must distinguish a known failure from an unknown outcome. Configure connection and response timeouts to bound network waits; a removal timeout follows the verification or safe-retry path described above.

Execution is **at least once**: a target may complete removal before a worker crashes while saving the result. Therefore, repeated removal must use the same logical request identifier and be safe under the target's contract. If safe repetition and reliable verification are both unavailable, request manual reconciliation.

An access check showing that access remains does not prove the original request stopped processing. Another removal request is issued only when safe.

Retry policies have a removal-attempt limit and an overall processing deadline. Exhausting removal attempts does not prevent checking an already accepted operation until the deadline. A failed status check retries the check, not the removal request.

When recovery is exhausted, record FAILED and whether removal remains unconfirmed. The deadline stops further automatic actions; it does not cancel processing already accepted by the target. Authorized operators can investigate and resume work or record verified completion, preserving the audit history.

5. Auditability and Operational Visibility

The decision and attempt records must answer:

- Who requested removal, for which review item and access assignment?
- When was it requested, attempted, and confirmed?
- What happened during each attempt?
- What evidence supports completion?
- Did an operator intervene, and why?

Retries preserve previous attempt outcomes. Operator actions record actor, timestamp, reason, and supporting evidence. Audit data is access-controlled, retained according to policy, and excludes credentials and unnecessary sensitive response data.

Monitor queue age, completion latency, target errors, retries, and overdue jobs. Alert on sustained failures and jobs requiring intervention.

The UI should distinguish **removal pending**, **removal confirmed**, and **requires attention**. It reports completion only after confirmation has been persisted.

6. Proposed Java Interface Changes and Tradeoffs

Domain model and repositories

Keep `AccessRight` as the entitlement definition. Introduce an `AccessAssignment` linking an account, right, and target; revoking one assignment must not modify the shared entitlement.

Add persistence for decisions, jobs, and attempts. Repository operations must support transactional job creation, atomic claims, and ownership-checked updates. Use explicit missing-record handling, such as `Optional<AccessRight>`.

DecisionService

Accept a request containing the review item, decision, and deduplication ID. Obtain the reviewer identity from the authenticated context.

```
DecisionReceipt decide(DecisionRequest request);
```

Persist the decision and, for revocation, its job in one transaction. Return the decision ID and optional job ID/status. Do not call the target from this transaction.

RevocationService

Replace the best-effort call and swallowed exception with durable job processing:

```
void process(ClaimedRevocationJob job);
```

Execute the job's next action, classify the outcome, and persist the attempt and next state. Unexpected failures remain observable, and unfinished work remains recoverable.

TargetSystemClient

Replace the void return type with an explicit result:

```
RemovalResult removeAccess(RemovalRequest request);
```

RemovalRequest identifies the target access and stable request key. RemovalResult distinguishes confirmed completion from pending processing and includes an operation ID when available.

Adapters expose supported status-query or access-verification operations, translate errors into the categories above, and configure bounded network timeouts. A successful transport response counts as completed removal only when the target contract guarantees it.

Implementation and tradeoff

A practical implementation would use Java, Spring Boot, PostgreSQL, and a scheduled worker. Spring scheduling triggers polling, while PostgreSQL provides durable work storage.

A database-backed queue keeps the decision and job atomic with limited infrastructure. Its costs are polling and implementing claims, leases, and retry scheduling. A broker could be introduced if delivery scale or integration needs justify it; a transactional outbox would then connect database changes to message publication reliably.

For this scope, the database-backed approach provides the required durability, recovery, and auditability without additional messaging infrastructure.

7. Train of thought and use of AI on this document

I recognized the nature of the problem, since I worked on a very similar real-world scenario a couple of years back. We needed to implement a Loyalty system in my previous job at Clara Card. The calculation of the total available points to be redeemed for an account in either cashback or fee discounts was a heavy-load process. It could be triggered by several people on the client side, and we needed to keep the process idempotent, resilient to app crashes (sometimes due to either unreliable infrastructure or heavy-traffic), and auditable, since we needed to know whether or not the process completed and what was its outcome.

Version 1 of that Loyalty system used some of the same principles seen here: a job queue backed by a database, workers polling, and asynchronous processes that were not predictable and required confirmation in order to mark the process as complete. Eventually, the system evolved and was migrated into an outbox pattern, since it ended up scaling and doing a lot of additional stuff. But the initial version got the product unstuck and the customers satisfied.

For this document creation, I used ChatGPT to bounce-off ideas and polish the solution writing and formatting. Diagram was done with code using Mermaid.